

PRUEBA

Documentación Técnica

Generado: 04/08/2026 03:15 UTC

AutoDocs AI

Tabla de Contenido

Tabla de Contenido	2
Documentación del Proyecto	4
Resumen Ejecutivo	4
Stack Tecnológico	4
Estadísticas del Proyecto	4
2. Documentación de Arquitectura y Diseño	5
2.1 Estructura de Directorios	5
2.2 Distribución por Capas	5
2.3 Distribución de Lenguajes	5
2.4 Diagrama de Infraestructura	6
2.5 Diagrama de Casos de Uso	6
2.6 Diagrama de Clases	7
2.7 Patrones Detectados	7
2.8 Insights de Arquitectura (IA)	7
3. Documentación Técnica de Código	7
3.1 Guía de Configuración del Entorno de Desarrollo	8
3.2 Documentación de API	8
3.3 Estado de Comentarios y Docstrings	12
3.4 Planes de Prueba (Testing)	14
4. Documentación de Clases	15
`User`	15
5. Documentación de Funciones	16

Módulo: `controllers/userController.py`	16
Módulo: `models/User.py`	17
Módulo: `routes/user.py`	18
6. Modelos de Datos	23
Esquema General Inferido	23
7. Guía de Despliegue y Mantenimiento	23
7.1 Requisitos del Sistema	23
7.2 Instalación Local	23
7.3 Despliegue con Docker	24
7.4 Variables de Entorno Recomendadas	24
7.5 Checklist de Despliegue	25
7.6 Registro de Cambios (Changelog)	25
8. Reporte de Calidad del Código	25
8.1 Análisis de Métricas	25
8.2 Funciones con Alta Complejidad Ciclomática	26
8.3 Recomendaciones	26
8.4 Errores de Análisis	26
9. Análisis de Datos del Proyecto	26
9.1 Tipos y Fuentes de Datos Detectados	26
9.2 Esquema de Data Warehouse Inferido	26
9.3 Reporte de Calidad de Datos	26
9.4 Minería de Datos Aplicada al Código	27
9.5 Pipeline ETL Utilizado	28

prueba

Documentación Técnica · AutoDocs AI · 04/08/2026

8	10	1	4
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 82/100

Documentación del Proyecto

Generado automáticamente por AutoDocs AI — 2026-08-04 03:15 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 8 archivos fuente, 10 funciones, 1 clases y 4 endpoints de API documentados. El score de calidad del código es **82/100**.

Stack Tecnológico

Lenguaje	Cantidad
Python	8 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	8
Funciones	10
Clases	1
Endpoints API	4
Score de calidad	82/100
Funciones documentadas	4 / 10
Clases documentadas	0 / 1

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/  
■■■ (raíz)/  
■■■ controllers/  
■■■ models/  
■■■ routes/
```

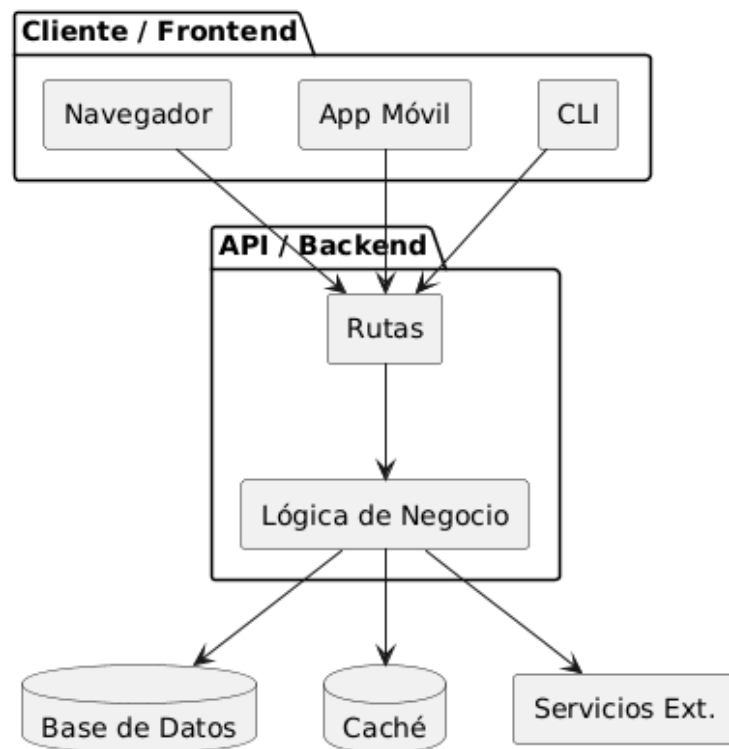
2.2 Distribución por Capas

- **(raíz)/** — 6 archivos [.env(1), .md(1), .py(2), .gitignore(1), .txt(1)]
- **controllers/** — 2 archivos [.py(2)]
- **models/** — 2 archivos [.py(2)]
- **routes/** — 2 archivos [.py(2)]

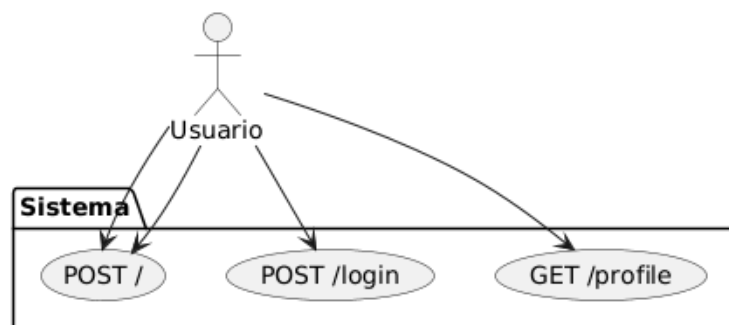
2.3 Distribución de Lenguajes

- Python: 8 archivos (100%)

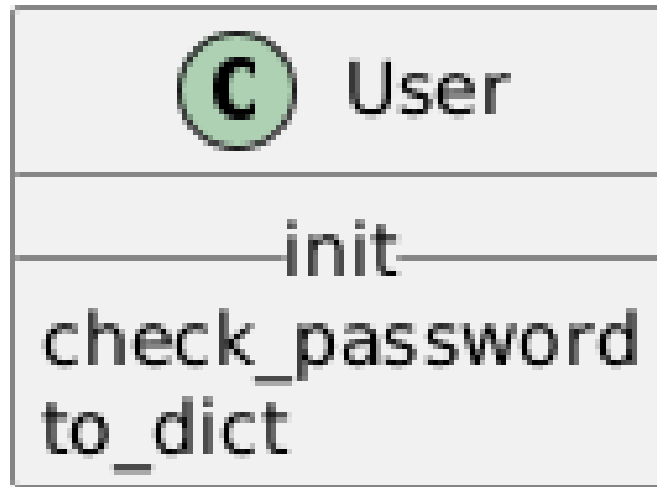
2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso



2.6 Diagrama de Clases



2.7 Patrones Detectados

- **API REST:** 4 endpoints detectados (Flask)
- **Programación Orientada a Objetos:** constructores detectados

2.8 Insights de Arquitectura (IA)

Insights Arquitectónicos

1. **Patrones arquitectónicos probables:** Monolito, debido a la baja cantidad de clases y funciones.
2. **Buenas prácticas detectadas:** Uso de un lenguaje principal (Python) y una estructura organizada (4 endpoints).
3. **Áreas de mejora sugeridas:** Incrementar la modularidad con más clases y funciones para mejorar la mantenibilidad.
4. **Complejidad del proyecto:** Baja, debido a la cantidad limitada de elementos.
5. **Recomendaciones de escalabilidad:** Considerar un patrón de arquitectura MVC o Microservicios para permitir una mayor flexibilidad y escalabilidad a medida que el proyecto crezca.

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: 4

Módulo: `routes/user.py`

[GET] GET /

- **Framework:** Flask
- **Archivo:** `routes/user.py`

Descripción del Endpoint

Propósito: El propósito de este endpoint es proporcionar información básica sobre el proyecto o la aplicación. Es como una página de bienvenida que te dice "¡Hola! Estoy aquí y estoy funcionando".

Funcionalidad: Cuando alguien visita esta dirección en la web, el endpoint envía una solicitud al servidor y el servidor responde con información sobre el proyecto. Es como pedir un menú en un restaurante: pides el menú y el mesero te trae una lista de opciones.

Información necesaria: Para que funcione, no necesitas proporcionar ninguna información adicional. Solo necesitas visitar la dirección en la web y listo. Es como ir a una página web: solo necesitas escribir la dirección y presionar Enter.

Resultado: El resultado es una respuesta del servidor con información sobre el proyecto. Puede ser un mensaje de bienvenida, una lista de opciones o simplemente un mensaje que dice "¡Todo está funcionando correctamente!". Es como recibir el menú en el restaurante: te dan una lista de opciones y puedes elegir lo que quieres.

Uso en la vida real: En la vida real, este tipo de endpoint se usa para verificar que una aplicación o un proyecto esté funcionando correctamente. Es como hacer una prueba de sonido antes de un concierto: quieres asegurarte de que todo esté funcionando bien antes

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

[POST] **POST** /

- **Framework:** Flask
- **Archivo:** `routes/user.py`

Descripción del Endpoint

Propósito: Este endpoint es como una puerta de entrada para crear nuevos usuarios en un sistema. Su propósito es recibir información sobre un nuevo usuario y agregarlo a la base de datos del sistema.

Funcionalidad: Cuando se envía una solicitud a este endpoint, lo que hace es tomar la información que se le proporciona (como el nombre, correo electrónico y contraseña del usuario) y la utiliza para crear un nuevo usuario en el sistema. Es como llenar un formulario para registrarse en una página web.

Información necesaria: Para que este endpoint funcione, necesita recibir cierta información básica sobre el usuario que se quiere crear. Esta información puede incluir:

- Nombre del usuario
- Correo electrónico
- Contraseña
- Otros datos relevantes para la creación del usuario

Resultado: Una vez que el endpoint procesa la información, devuelve un resultado que indica si el usuario se creó con éxito o si hubo algún error. Esto puede ser algo como un mensaje que dice "Usuario creado con éxito" o un código de error que indica qué salió mal.

Uso en la vida real: En la vida real, este tipo de endpoint se utiliza en situaciones como:

- Cuando te registras en una página web o aplicación para crear una cuenta de usuario.
- Cuando un administrador de un sistema crea nuevas cuentas de usuario para empleados o clientes.
- Cuando se necesita

Parámetros esperados:

- **body** (JSON, requerido): Cuerpo de la solicitud
- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

[POST] **POST** /login

- **Framework:** Flask
- **Archivo:** `routes/user.py`

Descripción del Endpoint de Inicio de Sesión

Propósito: El propósito de este endpoint es permitir a los usuarios iniciar sesión en una aplicación o sitio web de manera segura.

Funcionalidad: Cuando un usuario intenta acceder a una cuenta, este endpoint se encarga de verificar si la información proporcionada (como el nombre de usuario y la contraseña) es correcta. Si la información es válida, el endpoint permite al usuario acceder a su cuenta.

Información necesaria: Para que este endpoint funcione, necesita recibir la siguiente información:

- El nombre de usuario o correo electrónico del usuario.
- La contraseña del usuario.

Resultado: Si la información proporcionada es correcta, el endpoint devuelve un resultado que indica que el inicio de sesión ha sido exitoso. Esto puede incluir un token de acceso o una sesión iniciada, lo que permite al usuario acceder a áreas restringidas de la aplicación o sitio web.

Uso en la vida real: En la vida real, este tipo de endpoint se utiliza en casi todas las aplicaciones y sitios web que requieren que los usuarios inicien sesión para acceder a sus cuentas o a contenido protegido. Por ejemplo:

- Cuando inicias sesión en tu cuenta de correo electrónico, como Gmail o Outlook.
- Cuando accedes a tu cuenta bancaria en línea para revisar tus estados de cuenta o realizar transacciones.
- Cuando te conectas a redes

Parámetros esperados:

- **body** (JSON, requerido): Cuerpo de la solicitud
- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

[GET] **GET /profile**

- **Framework:** Flask
- **Archivo:** **routes/user.py**

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 10

- [OK] Documentadas: 4 (40%)
- [X] Sin documentar: 6 (60%)

Visualización de documentación:

[■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■] 40% documentado

Ejemplo de docstrings detectados:

```
def index():
    """
    Obtener todos los usuarios
    """
    tags:
    - Usuarios
    responses:
    200:
        description: Lista de usuarios
        """
        ...
        """
```

```
def user_store():
    """
    Crear un usuarios
    """
    tags:
        - Usuarios
```

```

parameters:
  - in: body
    name: user
    schema:
      type: object
      required:
        - name
        - email
        - password
      properties:
        name:
          type: string
          example: "Juan Pérez"
        email:
          type: string
          example: "juan@example.com"
        password:
          type: string
          example: "123456"
responses:
  201:
    description: Usuario creado
    ""
    ...

```

```

def login():
    """
    Iniciar sesión
    ---
tags:
  - Autenticación
parameters:
  - in: body
    name: credentials
    schema:
      type: object
      required:
        - email
        - password
      properties:
        email:
          type: string
          example: "juan@example.com"
        password:
          type: string
          example: "123456"
responses:

```

```
200:
    description: Login exitoso
    """
    ...
```

Funciones que requieren documentación:

- `get_all_users()` en `controllers/userController.py`
- `create_user(name, email, password)` en `controllers/userController.py`
- `login_user(email, password)` en `controllers/userController.py`
- `__init__(self, name, email, password)` en `models/User.py`
- `check_password(self, password)` en `models/User.py`
- ... y 1 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

Escenario	Tipo	Prioridad
GET / — respuesta 200	Integración	Alta
GET / — sin auth (401)	Seguridad	Alta

POST / — respuesta 200	Integración	Alta
POST / — sin auth (401)	Seguridad	Alta
POST /login — respuesta 200	Integración	Alta
POST /login — sin auth (401)	Seguridad	Alta
GET /profile — respuesta 200	Integración	Alta
GET /profile — sin auth (401)	Seguridad	Alta

4. Documentación de Clases

Total: **1 clases**

`User`

- Archivo: `models/User.py`
- Línea: 4
- Hereda de: `db.Model`
- Métodos: 3
- Documentada: [X] No

La Clase User: Un Administrador de Información de Usuarios

Propósito de la clase:

La clase User es como un archivo donde se almacenan y organizan los datos de los usuarios que interactúan con un sistema o aplicación. Su propósito es mantener y gestionar la información de cada usuario de manera segura y eficiente.

Representación en el sistema:

Imagina un fichero de empleados en una oficina. Cada empleado tiene un registro con su nombre, dirección, teléfono, etc. De manera similar, la clase User es como un fichero digital que almacena la información de cada usuario que utiliza el sistema. Esta información puede incluir nombres, correos electrónicos, contraseñas (protegidas, por supuesto), y otros datos relevantes.

Funcionalidades principales:

La clase User puede hacer varias cosas importantes:

- **Crear un nuevo usuario:** Cuando alguien se registra en el sistema, la clase User crea un nuevo "fichero" para esa persona y almacena su información.

- **Verificar contraseñas:** Cuando un usuario intenta acceder al sistema, la clase User verifica si la contraseña que ingresó es correcta, asegurando que solo los usuarios autorizados puedan acceder.
- **Convertir información en un formato útil:** La clase User puede tomar la información del usuario y convertirla en un formato que el sistema pueda

Métodos:

- `__init__()`
 - `check_password()`
 - `to_dict()`
-

5. Documentación de Funciones

Total: **10 funciones**

Módulo: `controllers/userController.py`

`get_all_users`

- Parámetros: –
- Async: —
- Complejidad: [LOW] 4
- Documentada: [X]

La función "get_all_users" sirve para obtener una lista de todos los usuarios registrados en un sistema. Para funcionar, no necesita datos adicionales. El resultado es una lista de usuarios. En la vida real, se usa en aplicaciones que requieren gestionar usuarios, como redes sociales o plataformas de aprendizaje. Esta función es importante porque permite administrar y monitorear a los usuarios del sistema de manera eficiente.

`create_user`

- Parámetros: `name`, `email`, `password`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

La función "create_user" sirve para crear un nuevo usuario en un sistema. Para funcionar, necesita saber el nombre, correo electrónico y contraseña

del usuario. Una vez que se proporcionan estos datos, la función crea una cuenta de usuario nueva. En la vida real, se usa para registrarse en sitios web o aplicaciones. Es importante porque permite a las personas acceder a servicios o contenido restringido. En el sistema, es fundamental para gestionar las cuentas de usuario y asegurar la seguridad de la información.

`login_user`

- Parámetros: `email`, `password`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

La función "login_user" permite a los usuarios acceder a un sistema o plataforma en línea. Necesita el correo electrónico y la contraseña para funcionar. Una vez que se proporcionan estos datos, la función verifica si son correctos y produce un resultado que indica si el acceso es permitido o no. En la vida real, se usa para acceder a cuentas en línea, como redes sociales o sitios web de compras. Esta función es crucial en el sistema porque garantiza la seguridad y privacidad de los usuarios, permitiéndoles acceder a sus cuentas de manera segura.

Módulo: ``models/User.py``

`__init__`

- Parámetros: `self`, `name`, `email`, `password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función `__init__` sirve para crear un nuevo usuario. Necesita saber el nombre, correo electrónico y contraseña del usuario. Con esta información, crea un perfil de usuario. Se utiliza en aplicaciones donde los usuarios necesitan registrarse, como redes sociales o plataformas de compras en línea. Esta función es crucial en el sistema porque permite la creación de perfiles de usuario, lo que es esencial para la autenticación y la personalización de la experiencia del usuario. En resumen, ayuda a darle una identidad a cada usuario en la plataforma.

`check_password`

- Parámetros: `self`, `password`
- Async: —
- Complejidad: [LOW] 1

- Documentada: [X]

La función "check_password" comprueba si una contraseña es correcta. Necesita saber quién está intentando acceder (representado por "self") y la contraseña que se está intentando usar. Luego, verifica si la contraseña coincide con la almacenada. Si coincide, devuelve un resultado positivo; de lo contrario, devuelve un resultado negativo. Se utiliza en situaciones como el inicio de sesión en una cuenta en línea, donde se necesita verificar la identidad del usuario. Esta función es crucial para mantener la seguridad y privacidad de los usuarios, ya que evita el acceso no autorizado a sus cuentas. En resumen, es una herramienta esencial para proteger la información personal en sistemas en línea.

to_dict

- Parámetros: `self`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

Módulo: ``routes/user.py``

index

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Obtener todos los usuarios

tags:

- Usuarios

responses:

200:

description: Lista de usuarios

user_store

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1

- Documentada: [OK]

Crear un usuarios

tags:

- Usuarios

parameters:

- in: body

name: user

schema:

type: object

required:

- name
- email
- password

properties:

name:

type: string

example: "Juan Pérez"

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

201:

description: Usuario creado

login

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1

- Documentada: [OK]

Iniciar sesión

tags:

- Autenticación

parameters:

- in: body

name: credentials

schema:

type: object

required:

- email
- password

properties:

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

200:

description: Login exitoso

profile

- Parámetros: —
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

Obtener perfil del usuario autenticado

tags:

- Usuarios

responses:

200:

description: Datos del usuario

``index()``

Ubicación: `routes/user.py`

Obtener todos los usuarios

tags:

- Usuarios

responses:

200:

description: Lista de usuarios

``user_store()``

Ubicación: `routes/user.py`

Crear un usuarios

tags:

- Usuarios

parameters:

- in: body

name: user

schema:

type: object

required:

- name

- email

- password

properties:

name:

type: string

example: "Juan Pérez"

email:
type: string
example: "juan@example.com"
password:
type: string
example: "123456"
responses:
201:
description: Usuario creado

``login()``

Ubicación: `routes/user.py`

Iniciar sesión

tags:
- Autenticación
parameters:
- in: body
name: credentials
schema:
type: object
required:
- email
- password
properties:
email:
type: string
example: "juan@example.com"
password:
type: string
example: "123456"
responses:

200:

description: Login exitoso

``profile()``

Ubicación: `routes/user.py`

Obtener perfil del usuario autenticado

tags:

- Usuarios

responses:

200:

description: Datos del usuario

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

- All Users
- User

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- Lenguaje principal: Python
- Archivos detectados: 8

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>
```

```
# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```


7.5 Checklist de Despliegue

- ☐ Variables de entorno configuradas
- ☐ Base de datos migrada
- ☐ Tests ejecutados y pasando
- ☐ Logs configurados
- ☐ HTTPS habilitado en producción
- ☐ Backups programados
- ☐ Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-08-04

- Versión inicial analizada por AutoDocs AI
- Documentación técnica generada automáticamente
- Análisis de calidad del código completado

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 82/100 (Excelente [GREEN])

Distribución de Documentación:

- Funciones documentadas: 4/10 (40%)
- Clases documentadas: 0/1

Análisis de Complejidad:

- Complejidad promedio: 1.8
- Complejidad máxima: 4
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 6 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 6
 - `get_all_users` en `controllers/userController.py`
 - `create_user` en `controllers/userController.py`
 - `login_user` en `controllers/userController.py`
 - `__init__` en `models/User.py`
 - `check_password` en `models/User.py`
 - ... y 1 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	1	Candidatas a simplificación
■ Baja	≤ 3	9	Complejidad aceptable

■ No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.

B) Funciones Sin Documentación

6 de 10 funciones carecen de docstring (60%).

Función	Archivo	Línea	Complejidad
`get_all_users`	`controllers/userController.py`	6	4
`create_user`	`controllers/userController.py`	13	3
`login_user`	`controllers/userController.py`	23	3
`__init__`	`models/User.py`	10	1
`check_password`	`models/User.py`	15	1
`to_dict`	`models/User.py`	18	1

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

■ No se detectaron funciones con nombres similares en archivos distintos.

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`routes/user.py`	4	1.2	100%	■ OK

`controllers/userController.py`	3	3.3	0%	■ OK
`models/User.py`	3	1.0	0%	■ OK

E) Patrones Frecuentes Adicionales (Análisis IA)

- **init**: 2 ocurrencias

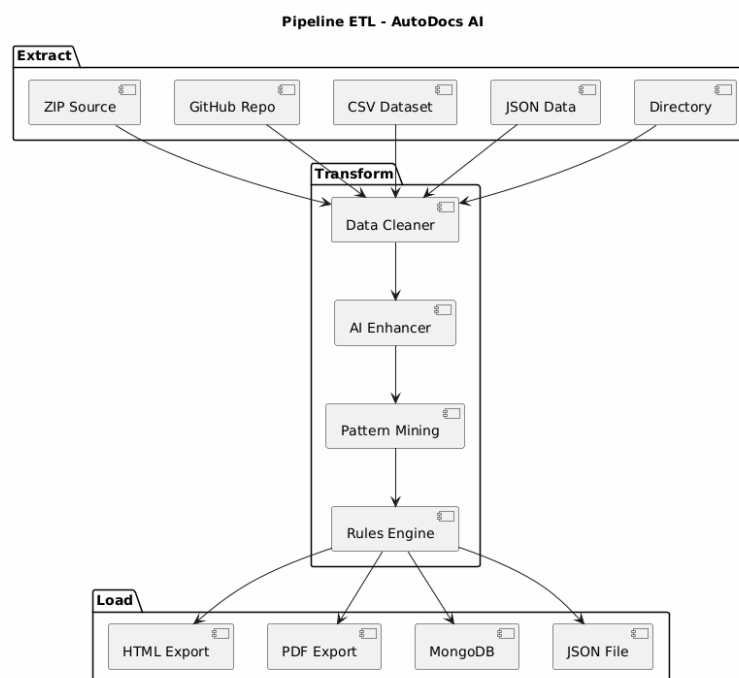
- Ejemplos: ['__init__', 'init']

- **user**: 5 ocurrencias

- Ejemplos: ['create_user', 'login_user', 'get_all_users', 'User', 'user_store']

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

- **Extract**: Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform**: Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load**: Generación de documentación en HTML, PDF o almacenamiento en MongoDB

